Operációs rendszerek

3. gyakorlat

Párhuzamos folyamatok,
külső programok futtatása

Folyamatok létrehozása

- **pid_t** típus
- pid_t **fork():**

  létrehoz egy gyerekfolyamatot, mely csak? a
  - PID (process identification) és
  - PPID (parent process identification) értékekben tér el a szülőtől.

  Értéke:
  - a szülő folyamatban:
    - a létrehozott gyerekfolyamat azonosítója
    - -1 (hiba).
  - a gyerek folyamatban: 0

Folyamatok kezelése

- **getpid:** a folyamat azonosítójának lekérdezése
- **getppid:** a folyamat szülőjének azonosítóját kérdezi le
- **wait:** felfüggeszti a folyamat végrehajtását, míg *bármelyik* gyerekfolyamat terminál
- **waitpid:** felfüggeszti a folyamat végrehajtását, míg *egy megadott* gyerekfolyamat *állapota* megváltozik
- **sleep**, **usleep**: felfüggeszti a folyamat végrehajtását a megadott ideig, vagy egy szignál érkezéséig (man sleep.3)

Szülő-gyerek PID-ek

- Gyerek:
  - fork(): 0
  - getpid: gyerek PID
  - getppid: szülő PID
- Szülő:
  - fork(): gyerek PID
  - getpid: szülő PID
  - getppid: bash (shell) PID

Gyerekből lekérdezhető a szülő PID-je, de fordítva ez nem lehetséges.

Szülőből csak akkor tudhatjuk egy gyerekének PID-jét, ha a fork visszatérési értékét eltároljuk.

Megoldatlan problémák

1. A folyamatok szinkronizálása
2. A folyamatok közötti adatcsere

Ezek megoldását később fogujk megismerni.

gy1/fork.c

- **fork():** létrehoz egy gyerekfolyamatot
- **perror:** errno szerinti hibaüzenet kiíratása
- **errno:** a legutóbb bekövetkezett hiba kódja
- **waitpid:** felfüggeszti a folyamat végrehajtását, míg *egy megadott* gyerekfolyamat *állapota* megváltozik

gy1/exec.c és system.c

- **execv, execl**: lecseréli az aktuális folyamatot egy külső programra
- **system**: létrehoz egy gyerekfolyamatot, és végrehajttat vele egy külső programot

Házi feladat

- 3. feladatsor megoldása (klettner.web.elte.hu → Oktatás → Operációs rendszerek → Gyakorló feladatok)